

Task -1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score,
                             f1_score, confusion_matrix,
                             ConfusionMatrixDisplay)
sns.set_style("whitegrid")
np.random.seed(42)

try:
    from google.colab import files
    print("Please upload 'data_banknote_authentication.txt'")
    uploaded = files.upload()
    file_path = list(uploaded.keys())[0]
except ImportError:
    # Not running in Colab (e.g. local testing) – set path manually
    file_path = "data_banknote_authentication.txt"

Please upload 'data_banknote_authentication.txt'

<IPython.core.display.HTML object>

Saving data_banknote_authentication.txt to
data_banknote_authentication.txt

# Task 1: Dataset Exploration
columns = ["Variance", "Skewness", "Curtosis", "Entropy", "Class"]
df = pd.read_csv(file_path, header=None, names=columns)

print("=" * 60)
print("TASK 1: DATASET EXPLORATION")
print("=" * 60)

print("\nFirst five samples:")
print(df.head())

print("\nDataset dimensions (rows, columns):", df.shape)

print("\nMissing values per column:")
print(df.isnull().sum())

print("\nDescriptive statistics:")
print(df.describe())
```

```
print("\nClass distribution:")
print(df["Class"].value_counts())
```

TASK 1: DATASET EXPLORATION

First five samples:

	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Dataset dimensions (rows, columns): (1372, 5)

Missing values per column:

```
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0
dtype: int64
```

Descriptive statistics:

	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Class distribution:

```
Class
0    762
1    610
Name: count, dtype: int64
```

TASK 2

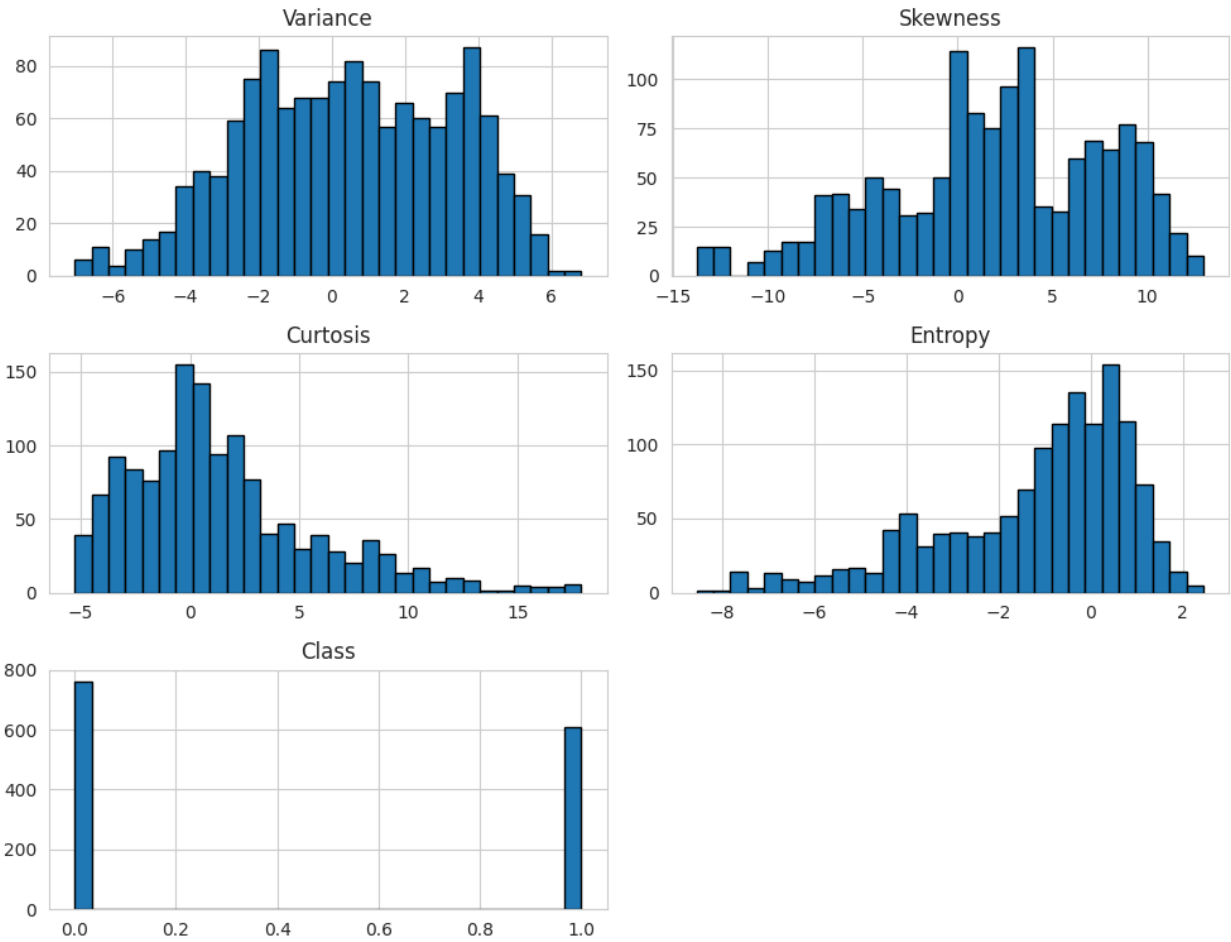
```
print("\n" + "=" * 60)
print("TASK 2: EXPLORATORY DATA ANALYSIS")
print("=" * 60)
```

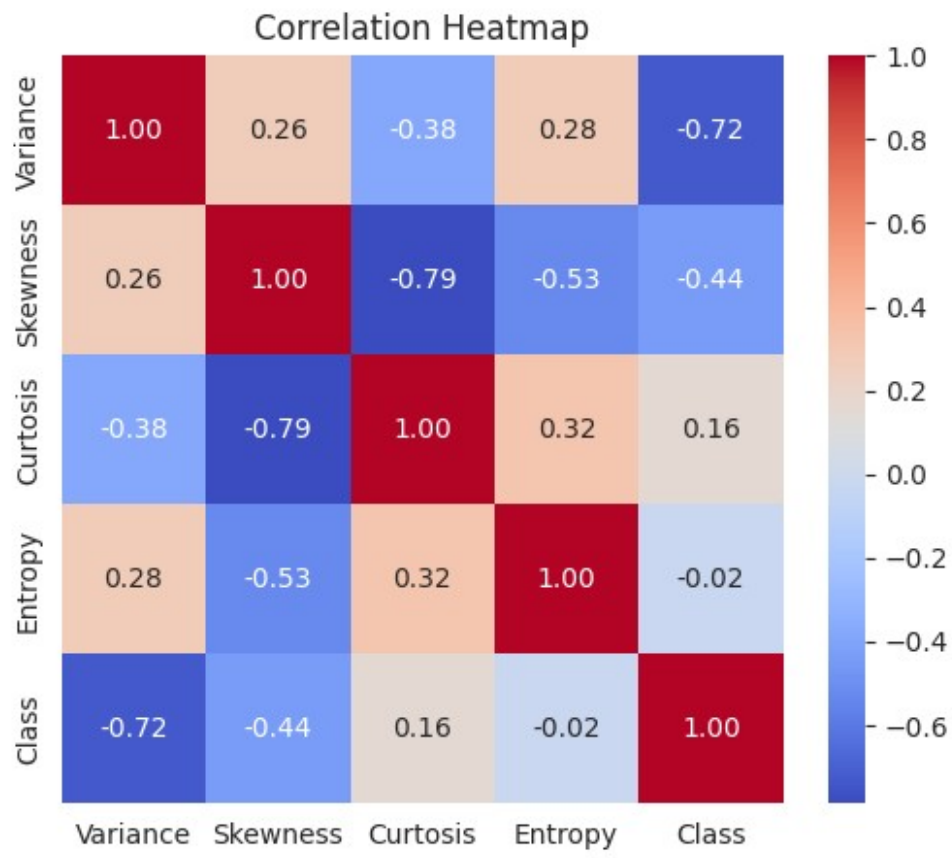
```
df.hist(figsize=(10, 8), bins=30, edgecolor="black")
plt.suptitle("Feature Histograms")
```

```
plt.tight_layout()
plt.show()
plt.figure(figsize=(6, 5))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
plt.figure(figsize=(6, 5))
sns.scatterplot(data=df, x="Variance", y="Skewness", hue="Class",
palette="Set1", alpha=0.7)
plt.title("Scatter Plot: Variance vs Skewness")
plt.show()
plt.figure(figsize=(10, 6))
df_melted = df.melt(id_vars="Class", value_vars=["Variance",
"Skewness", "Curtosis", "Entropy"])
sns.boxplot(data=df_melted, x="variable", y="value", hue="Class")
plt.title("Boxplots of Features by Class")
plt.show()
```

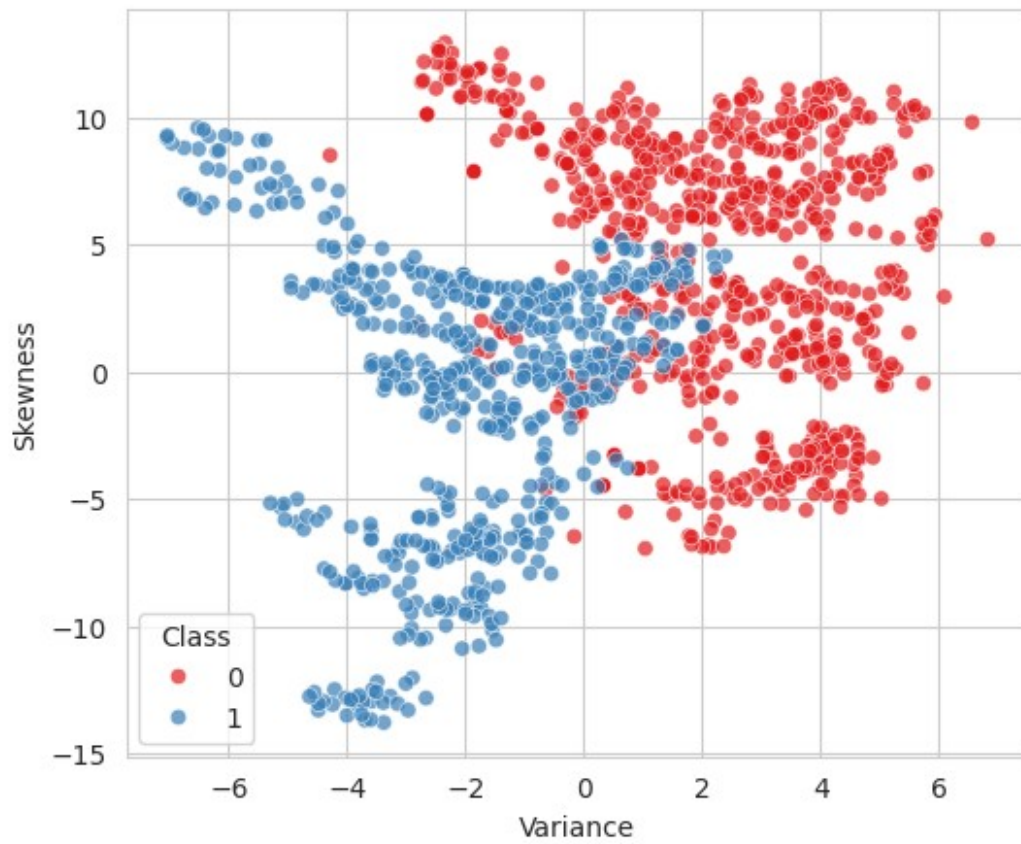
TASK 2: EXPLORATORY DATA ANALYSIS

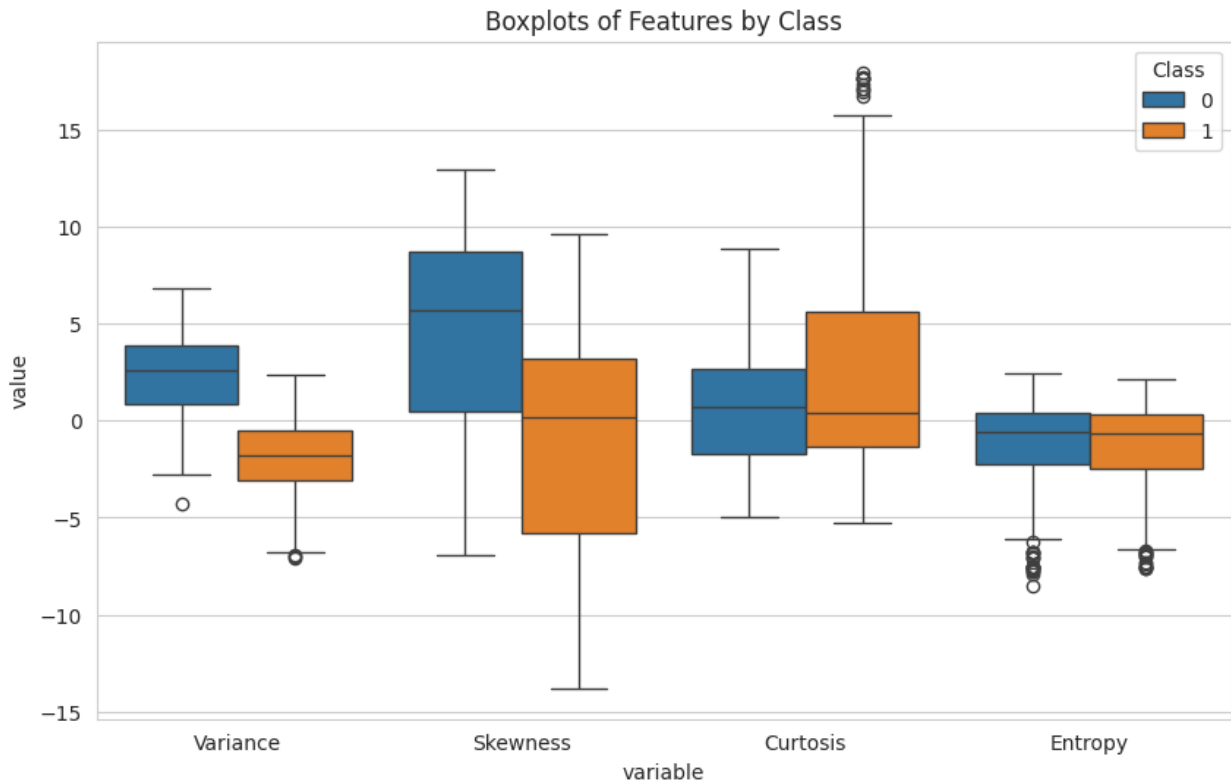
Feature Histograms





Scatter Plot: Variance vs Skewness





TASK 3

```
print("\n" + "=" * 60)
print("TASK 3: DATA PREPROCESSING")
print("=" * 60)
X = df[["Variance", "Skewness", "Kurtosis", "Entropy"]].values
y = df["Class"].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

```
=====
TASK 3: DATA PREPROCESSING
=====
Training set size: (1097, 4)
Testing set size: (275, 4)
```

TASK 4 and 5

```

print("\n" + "=" * 60)
print("TASK 4 & 5: PERCEPTRON IMPLEMENTATION & TRAINING")
print("=" * 60)

class Perceptron:
    """Single Layer Perceptron implemented from scratch."""

    def __init__(self, n_features, learning_rate=0.01, epochs=50):
        self.lr = learning_rate
        self.epochs = epochs

        self.weights = np.zeros(n_features)
        self.bias = 0.0
        self.errors_history = []
        self.weights_history = []
        self.bias_history = []

    @staticmethod
    def step_activation(z):
        """Step Activation Function:  $f(z) = 1$  if  $z \geq 0$  else  $0$ """
        return np.where(z >= 0, 1, 0)

    def forward(self, X):
        """Weighted sum + activation"""
        z = np.dot(X, self.weights) + self.bias
        return self.step_activation(z)

    def fit(self, X, y, verbose=True):
        n_samples = X.shape[0]

        for epoch in range(1, self.epochs + 1):
            misclassified = 0

            for idx in range(n_samples):
                xi = X[idx]
                target = y[idx]

                z = np.dot(xi, self.weights) + self.bias
                y_pred = self.step_activation(z)

                error = target - y_pred
                if error != 0:
                    misclassified += 1

                    self.weights += self.lr * error * xi
                    self.bias += self.lr * error

            self.errors_history.append(misclassified)
            self.weights_history.append(self.weights.copy())
            self.bias_history.append(self.bias)

            if verbose:

```



```

        print(f"Epoch {epoch:3d} | Misclassified:
{misclassified:4d} | "
              f"Weights: {np.round(self.weights, 4)} | Bias:
{round(self.bias, 4)}")

        if misclassified == 0:
            if verbose:
                print(f"\nConverged at epoch {epoch}.")
            break

    return self

def predict(self, X):
    return self.forward(X)

```

LEARNING_RATE = 0.01

EPOCHS = 50

```

model = Perceptron(n_features=X_train.shape[1],
learning_rate=LEARNING_RATE, epochs=EPOCHS)
model.fit(X_train, y_train, verbose=True)

```

=====

TASK 4 & 5: PERCEPTRON IMPLEMENTATION & TRAINING

=====

```

Epoch 1 | Misclassified: 59 | Weights: [-0.068 -0.0938 -0.0678 -
0.011 ] | Bias: -0.03
Epoch 2 | Misclassified: 25 | Weights: [-0.0775 -0.1042 -0.0811
0.0035] | Bias: -0.04
Epoch 3 | Misclassified: 20 | Weights: [-0.0802 -0.1127 -0.0917 -
0.0036] | Bias: -0.04
Epoch 4 | Misclassified: 19 | Weights: [-0.0883 -0.1196 -0.1001
0.0089] | Bias: -0.05
Epoch 5 | Misclassified: 20 | Weights: [-0.0991 -0.1295 -0.1037
0.0102] | Bias: -0.05
Epoch 6 | Misclassified: 17 | Weights: [-0.1027 -0.1277 -0.1134 -
0.001 ] | Bias: -0.06
Epoch 7 | Misclassified: 24 | Weights: [-0.1081 -0.1393 -0.1108 -
0.0094] | Bias: -0.06
Epoch 8 | Misclassified: 22 | Weights: [-0.117 -0.1357 -0.1319 -
0.0025] | Bias: -0.06
Epoch 9 | Misclassified: 20 | Weights: [-0.1176 -0.147 -0.1364 -
0.0062] | Bias: -0.06
Epoch 10 | Misclassified: 20 | Weights: [-0.1307 -0.1541 -0.125 -
0.006 ] | Bias: -0.06
Epoch 11 | Misclassified: 22 | Weights: [-0.1345 -0.1641 -0.1329 -
0.0018] | Bias: -0.06
Epoch 12 | Misclassified: 20 | Weights: [-0.1287 -0.16 -0.1472

```

```
0.0076] | Bias: -0.08
Epoch 13 | Misclassified: 16 | Weights: [-0.1343 -0.1627 -0.1458
0.0039] | Bias: -0.08
Epoch 14 | Misclassified: 24 | Weights: [-1.357e-01 -1.673e-01 -
1.549e-01 1.000e-04] | Bias: -0.08
Epoch 15 | Misclassified: 17 | Weights: [-0.1362 -0.1802 -0.1518 -
0.015 ] | Bias: -0.07
Epoch 16 | Misclassified: 21 | Weights: [-0.1464 -0.1877 -0.1507 -
0.0034] | Bias: -0.06
Epoch 17 | Misclassified: 19 | Weights: [-0.143 -0.1793 -0.1685 -
0.0169] | Bias: -0.07
Epoch 18 | Misclassified: 18 | Weights: [-0.1603 -0.1845 -0.1543 -
0.0013] | Bias: -0.07
Epoch 19 | Misclassified: 23 | Weights: [-0.1503 -0.1853 -0.1712 -
0.0101] | Bias: -0.08
Epoch 20 | Misclassified: 19 | Weights: [-0.1636 -0.1946 -0.1613 -
0.0064] | Bias: -0.07
Epoch 21 | Misclassified: 18 | Weights: [-0.1485 -0.1985 -0.1725 -
0.0064] | Bias: -0.09
Epoch 22 | Misclassified: 15 | Weights: [-0.1575 -0.1939 -0.1715 -
0.0254] | Bias: -0.08
Epoch 23 | Misclassified: 12 | Weights: [-0.1599 -0.1909 -0.1749 -
0.0174] | Bias: -0.08
Epoch 24 | Misclassified: 19 | Weights: [-0.1565 -0.1956 -0.1756 -
0.0168] | Bias: -0.09
Epoch 25 | Misclassified: 17 | Weights: [-0.1578 -0.1971 -0.1794 -
0.0215] | Bias: -0.08
Epoch 26 | Misclassified: 19 | Weights: [-0.1597 -0.1994 -0.1804 -
0.0162] | Bias: -0.09
Epoch 27 | Misclassified: 22 | Weights: [-0.1669 -0.1961 -0.1856 -
0.0184] | Bias: -0.09
Epoch 28 | Misclassified: 17 | Weights: [-0.1708 -0.2018 -0.184 -
0.0161] | Bias: -0.08
Epoch 29 | Misclassified: 19 | Weights: [-0.1728 -0.204 -0.1863 -
0.0122] | Bias: -0.09
Epoch 30 | Misclassified: 17 | Weights: [-0.1751 -0.2043 -0.1901 -
0.0196] | Bias: -0.08
Epoch 31 | Misclassified: 19 | Weights: [-0.1742 -0.2067 -0.1898 -
0.0116] | Bias: -0.09
Epoch 32 | Misclassified: 17 | Weights: [-0.1765 -0.207 -0.1935 -
0.0189] | Bias: -0.08
Epoch 33 | Misclassified: 18 | Weights: [-0.1787 -0.2104 -0.1912 -
0.0212] | Bias: -0.08
Epoch 34 | Misclassified: 15 | Weights: [-0.1692 -0.2152 -0.1926 -
0.0305] | Bias: -0.09
Epoch 35 | Misclassified: 12 | Weights: [-0.1708 -0.2136 -0.1935 -
0.0217] | Bias: -0.09
Epoch 36 | Misclassified: 17 | Weights: [-0.1738 -0.2179 -0.1937 -
0.0199] | Bias: -0.08
```

```

Epoch 37 | Misclassified: 19 | Weights: [-0.1694 -0.2308 -0.1887 -
0.0267] | Bias: -0.09
Epoch 38 | Misclassified: 16 | Weights: [-0.1748 -0.2241 -0.1929 -
0.0279] | Bias: -0.09
Epoch 39 | Misclassified: 12 | Weights: [-0.1772 -0.2211 -0.1963 -
0.0199] | Bias: -0.09
Epoch 40 | Misclassified: 18 | Weights: [-0.1948 -0.2244 -0.1853 -
0.0197] | Bias: -0.09
Epoch 41 | Misclassified: 20 | Weights: [-0.1912 -0.2341 -0.1912 -
0.0163] | Bias: -0.09
Epoch 42 | Misclassified: 16 | Weights: [-0.185 -0.2384 -0.1968 -
0.0137] | Bias: -0.09
Epoch 43 | Misclassified: 16 | Weights: [-0.1935 -0.2316 -0.2018 -
0.0254] | Bias: -0.09
Epoch 44 | Misclassified: 19 | Weights: [-0.1939 -0.224 -0.2157 -
0.0164] | Bias: -0.1
Epoch 45 | Misclassified: 18 | Weights: [-0.1864 -0.2389 -0.2117 -
0.0058] | Bias: -0.1
Epoch 46 | Misclassified: 15 | Weights: [-0.1942 -0.2323 -0.2175 -
0.0208] | Bias: -0.09
Epoch 47 | Misclassified: 16 | Weights: [-0.1987 -0.2279 -0.218 -
0.0179] | Bias: -0.09
Epoch 48 | Misclassified: 17 | Weights: [-0.1951 -0.2336 -0.2113 -
0.0119] | Bias: -0.1
Epoch 49 | Misclassified: 14 | Weights: [-0.2092 -0.2231 -0.2069 -
0.0255] | Bias: -0.1
Epoch 50 | Misclassified: 18 | Weights: [-0.1941 -0.2358 -0.2109 -
0.0195] | Bias: -0.1
<__main__.Perceptron at 0x7e3232c19b80>

```

TASK 6

```

print("\n" + "=" * 60)
print("TASK 6: MODEL EVALUATION")
print("=" * 60)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy   : {acc:.4f}")
print(f"Precision   : {prec:.4f}")
print(f"Recall      : {rec:.4f}")
print(f"F1-score    : {f1:.4f}")

```

```

print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=["Authentic (0)", "Forged (1)"])
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
plt.figure(figsize=(7, 5))
plt.plot(range(1, len(model.errors_history) + 1),
model.errors_history, marker="o", color="crimson")
plt.xlabel("Epoch")
plt.ylabel("Misclassified Samples")
plt.title("Training Error vs Epoch")
plt.grid(True)
plt.show()
weights_arr = np.array(model.weights_history)
plt.figure(figsize=(7, 5))
for i, fname in enumerate(["Variance", "Skewness", "Curtosis",
"Entropy"]):
    plt.plot(weights_arr[:, i], label=f"w_{fname}")
plt.xlabel("Epoch")
plt.ylabel("Weight Value")
plt.title("Weight Evolution over Epochs")
plt.legend()
plt.show()
plt.figure(figsize=(7, 5))
plt.plot(model.bias_history, color="green", marker="s")
plt.xlabel("Epoch")
plt.ylabel("Bias Value")
plt.title("Bias Evolution over Epochs")
plt.show()

```

TASK 6: MODEL EVALUATION

```

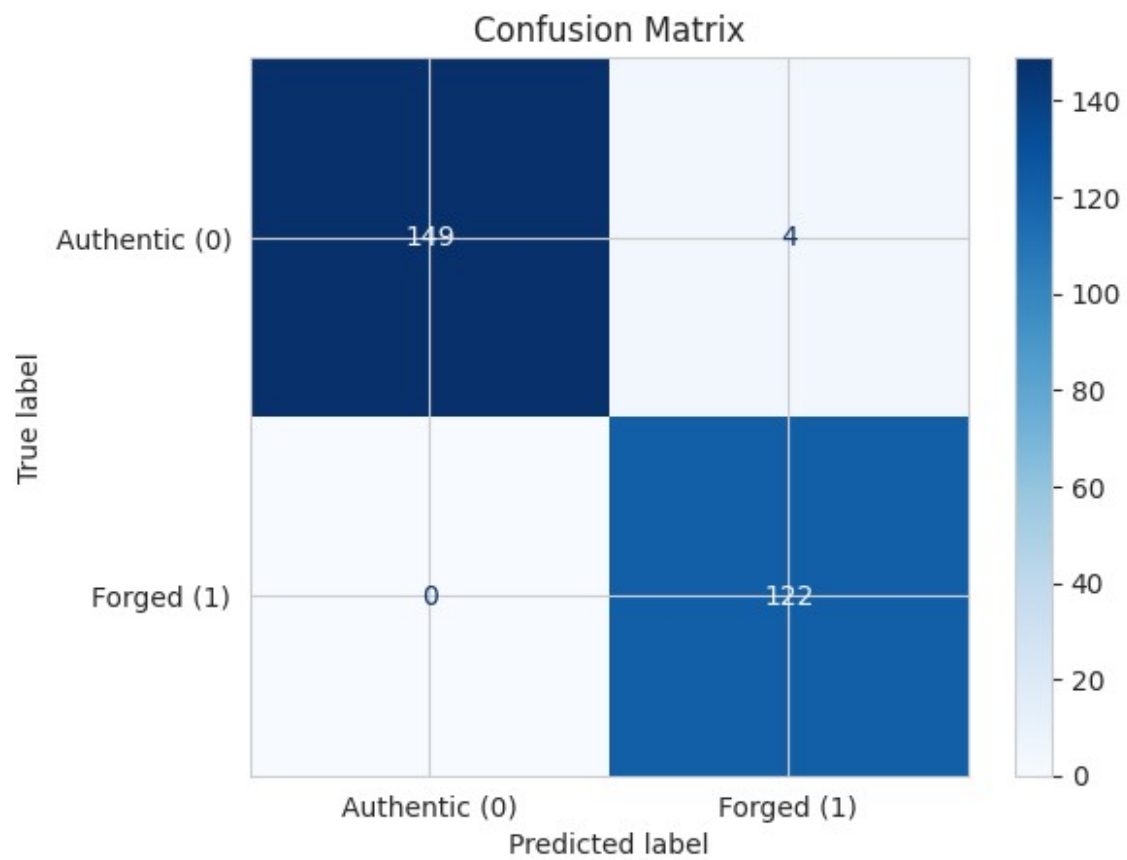
Accuracy : 0.9855
Precision : 0.9683
Recall : 1.0000
F1-score : 0.9839

```

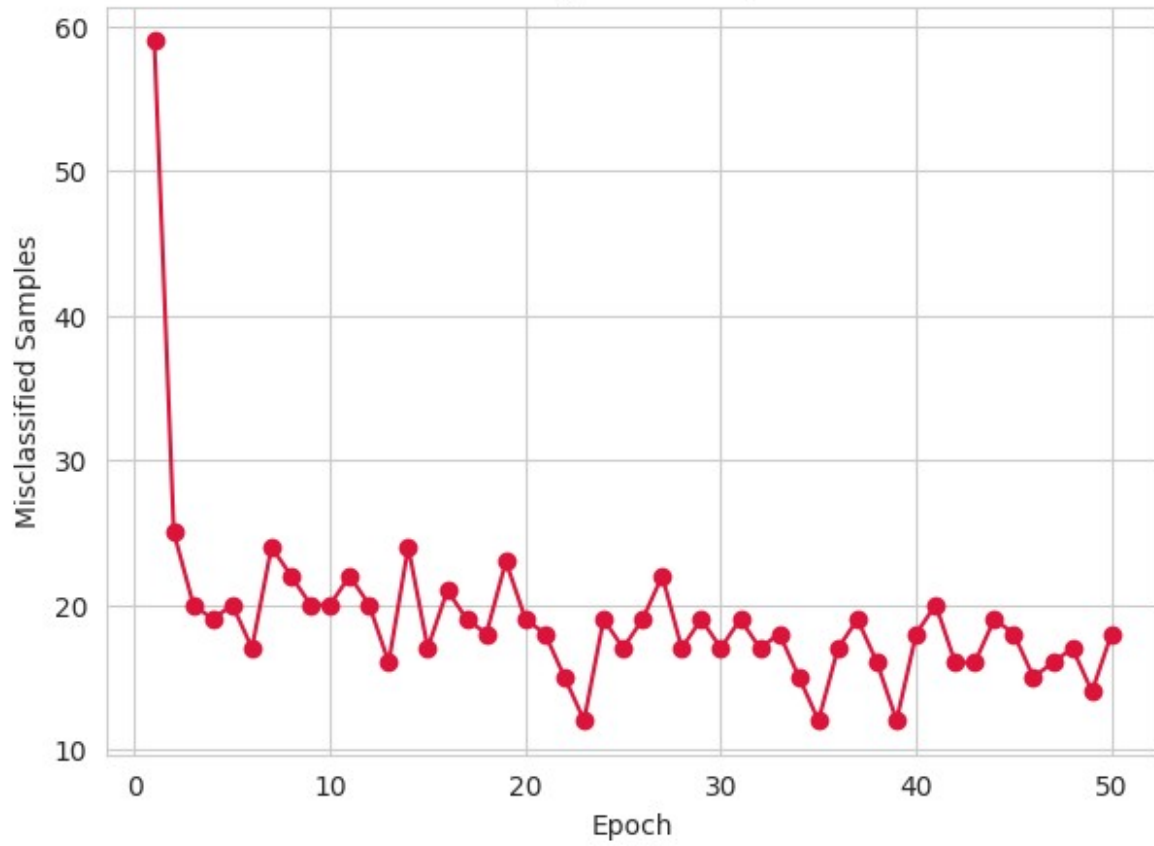
```

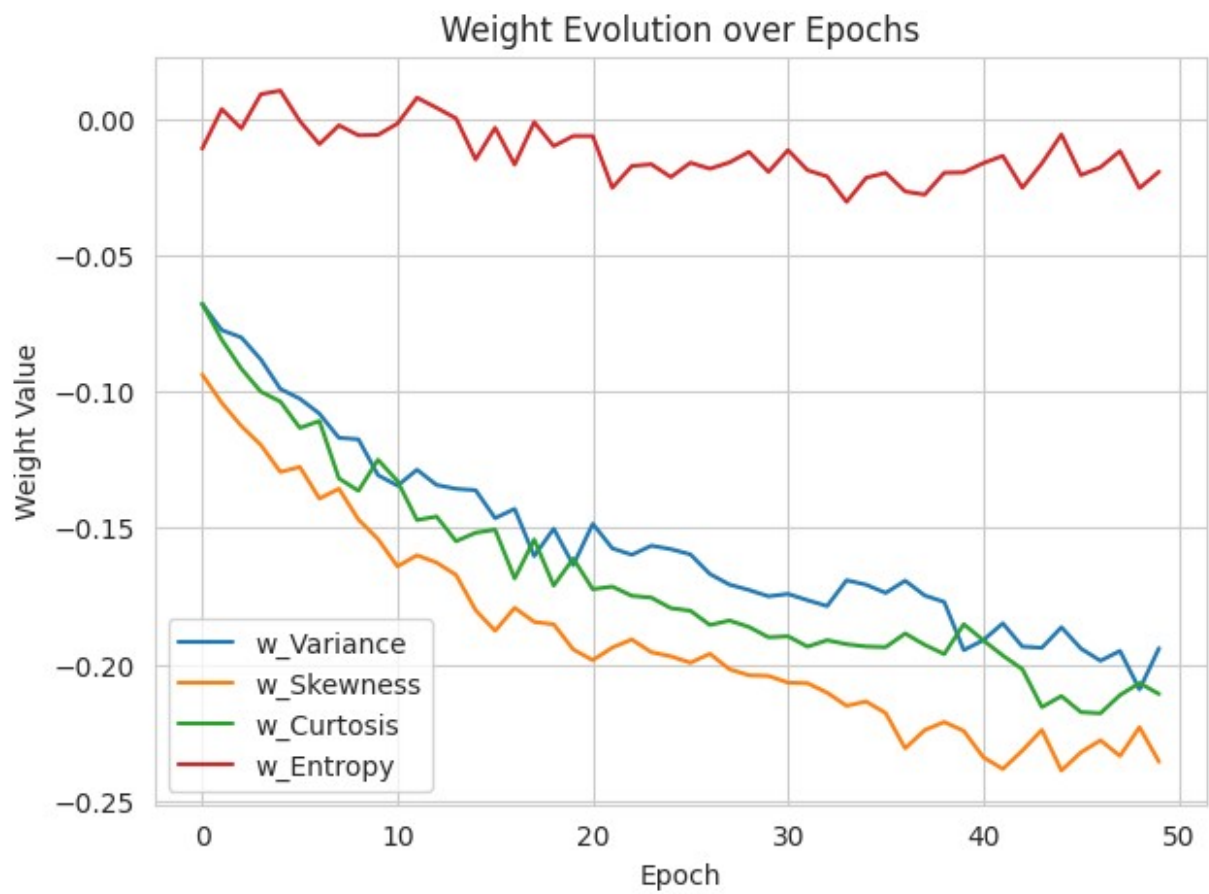
Confusion Matrix:
[[149  4]
 [ 0 122]]

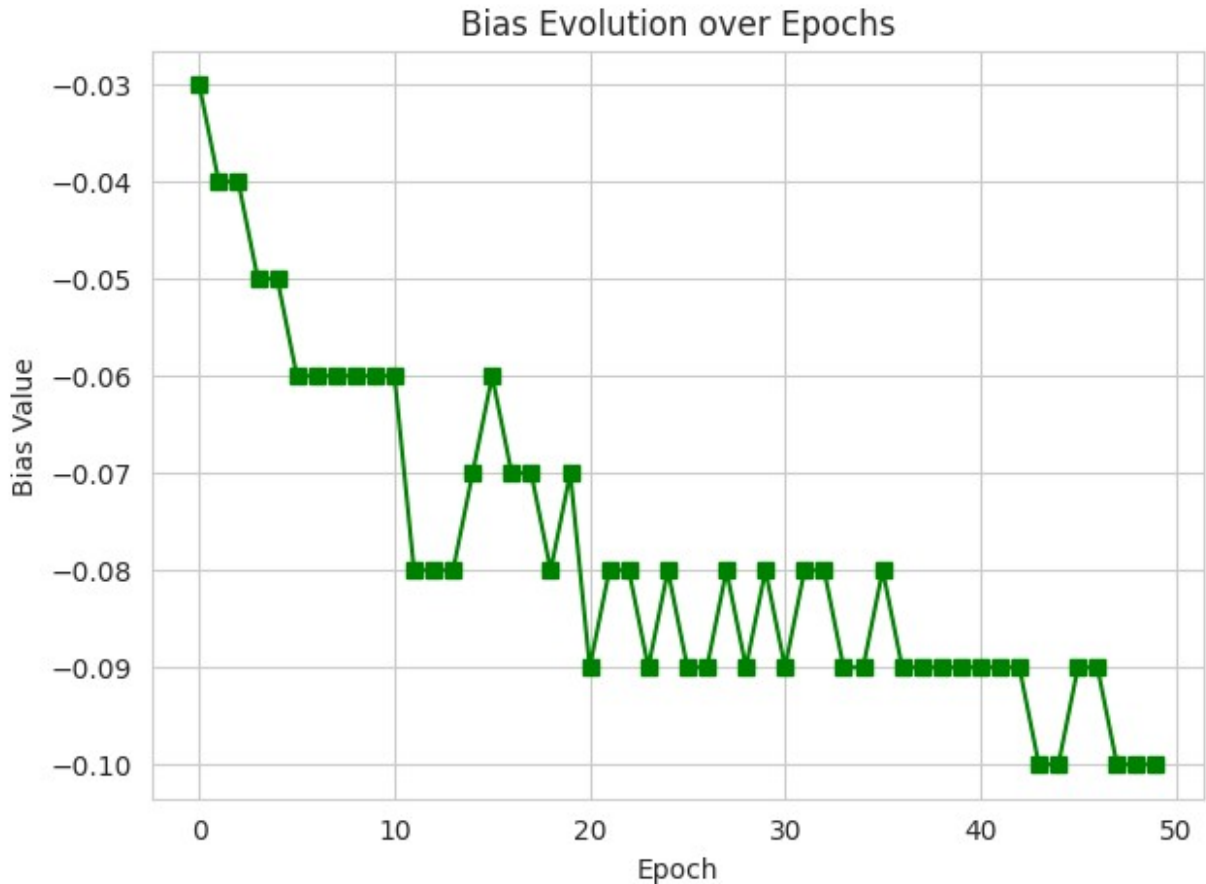
```



Training Error vs Epoch







TASK 7

```

print("\n" + "=" * 60)
print("TASK 7: LEARNING RATE COMPARISON")
print("=" * 60)

learning_rates = [0.001, 0.01, 0.1]
lr_results = {}

plt.figure(figsize=(8, 6))
for lr in learning_rates:
    m = Perceptron(n_features=X_train.shape[1], learning_rate=lr,
epochs=50)
    m.fit(X_train, y_train, verbose=False)
    lr_results[lr] = m

    y_pred_lr = m.predict(X_test)
    acc_lr = accuracy_score(y_test, y_pred_lr)
    print(f"Learning Rate = {lr:<6} | Epochs run =
{len(m.errors_history):3d} "
          f"| Final Misclassified = {m.errors_history[-1]:4d} | Test
Accuracy = {acc_lr:.4f}")

```

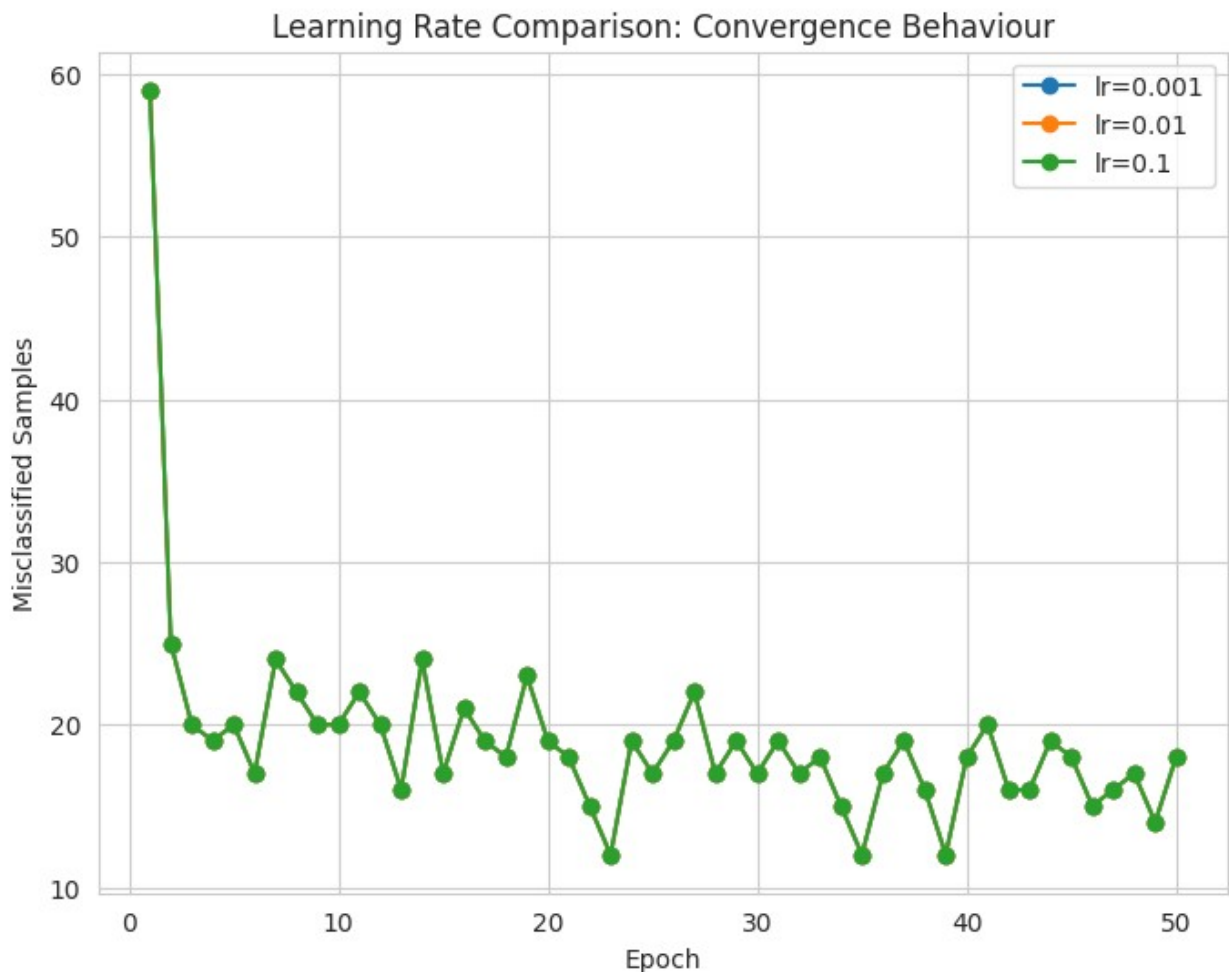


```
plt.plot(range(1, len(m.errors_history) + 1), m.errors_history,  
marker="o", label=f"lr={lr}")
```

```
plt.xlabel("Epoch")  
plt.ylabel("Misclassified Samples")  
plt.title("Learning Rate Comparison: Convergence Behaviour")  
plt.legend()  
plt.show()
```

TASK 7: LEARNING RATE COMPARISON

```
Learning Rate = 0.001 | Epochs run = 50 | Final Misclassified = 18  
| Test Accuracy = 0.9855  
Learning Rate = 0.01 | Epochs run = 50 | Final Misclassified = 18  
| Test Accuracy = 0.9855  
Learning Rate = 0.1 | Epochs run = 50 | Final Misclassified = 18  
| Test Accuracy = 0.9855
```



TASK 8

```
print("\n" + "=" * 60)
print("OPTIONAL: DECISION BOUNDARY (Variance vs Skewness)")
print("=" * 60)

X_2d = X_train[:, :2] # Variance, Skewness only
model_2d = Perceptron(n_features=2, learning_rate=0.01, epochs=50)
model_2d.fit(X_2d, y_train, verbose=False)

x_min, x_max = X_2d[:, 0].min() - 1, X_2d[:, 0].max() + 1
y_min, y_max = X_2d[:, 1].min() - 1, X_2d[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05), np.arange(y_min,
y_max, 0.05))
Z = model_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(7, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y_train, cmap="coolwarm",
edgecolor="k")
plt.xlabel("Variance (scaled)")
plt.ylabel("Skewness (scaled)")
plt.title("Decision Boundary: Variance vs Skewness")
plt.show()
```

```
=====
OPTIONAL: DECISION BOUNDARY (Variance vs Skewness)
=====
```

Decision Boundary: Variance vs Skewness

